U24AI022
KOMAL SAI JAKKA

ASSIGNMENT-4

Q1 Use the city map given in Q1 of assignment 1 and implement the best first search algorithm to reach your destination and compare your results in terms of the number of possible paths explored.

CODE:

```
#include<bits/stdc++.h>
using namespace std;

struct node{
int city;
int h;
vector<int> path;
};

class priorityqueue{
public:
vector<node> pq;
void push(node n){
pq.push_back(n);
}
node pop(){
int minindex=0;
for(int i=1;i<pq.size();i++){
if(pq[i].h<pq[minindex].h){
minindex=i;
}
}
node best=pq[minindex];
pq.erase(pq.begin()+minindex);
return best;
}
bool empty(){
return pq.empty();
}
};

int main(){
string cities[14]={
"chicago","detroit","cleveland","buffalo","syracuse","pittsburgh","newyork",
"philedalphia","boston","indianapolis","columbus","baltimore","portland","pr
ovidence"
};
int heuristic[14]={4,3,3,2,1,3,1,2,0,5,4,3,1,1};
vector<int> citymap[14];
```

```
citymap[0]={1,2,9};
citymap[1]={0,2,3};
citymap[2]={0,1,10,5};
citymap[3]={1,2,5,4};
citymap[4]={3,8,6};
citymap[5]={2,3,10,7,11};
citymap[6]={4,7,8,13};
citymap[7]={5,6,11};
citymap[8]={4,6,12,13};
citymap[9]={0};
citymap[10]={2,5};
citymap[11]={5,7};
citymap[12]={8};
citymap[13]={6,8};
int start=0;
int goal=8;
priorityqueue pq;
vector<bool> vis(12,false);
node startnode;
startnode.city=start;
startnode.h=heuristic[start];
startnode.path.push_back(start);
pq.push(startnode);
int exp=0;
cout<<"best first search using hop heuristic"<<endl;
while(!pq.empty()){
node curr=pq.pop();
exp++;
int currcity=curr.city;
if(vis[currcity]){
continue;
}
vis[currcity]=true;
if(currcity==goal){
cout<<"path is :"<<endl;
for(int i=0;i<curr.path.size();i++){
cout<<cities[curr.path[i]];
if(i!=curr.path.size()-1){
cout<<"→";
}
}
cout<<"\nexplored:"<<exp<<endl;
return 0;
}
for(int neigh : citymap[currcity]){
if(!vis[neigh]){
node next;
next.city=neigh;
next.h=heuristic[neigh];
next.path=curr.path;
next.path.push_back(neigh);
pq.push(next);
```

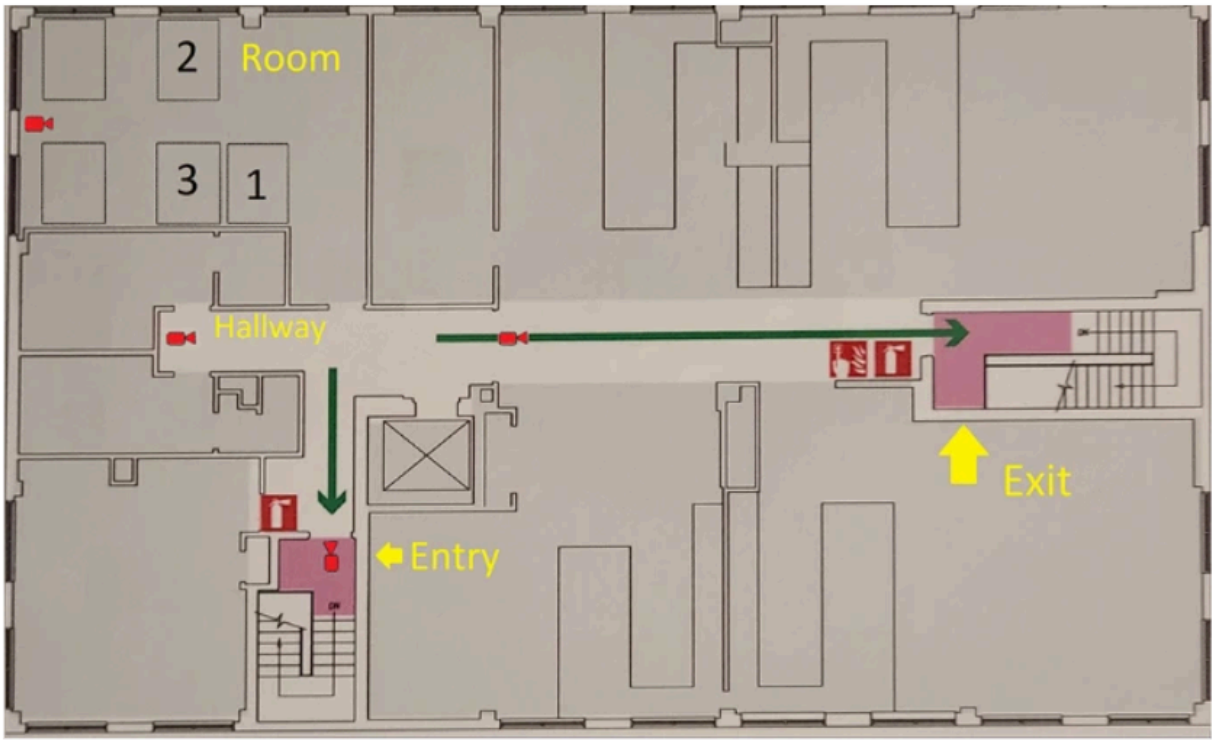
```
}
}
}
cout<<"goal not found"<<endl;
return 0;
}
```

OUTPUT:

best first search using hop heuristic
path is :
chicago->detroit->buffalo->syracuse->boston
explored:5

Q2 Consider a building floor plan as given in figure. Design a robot guided evacuation planning using the best first search algorithm. Consider the following constraints:

- Allowed movements up, down, left and right
- Walls and rooms block movement
- Evacuation planning covers the route from entry to exit
- Design an evaluation cost function with proper justification for your algorithm



CODE:

```
#include<bits/stdc++.h>
using namespace std;

struct rule{
int id;
string cond;
string action;
```

```
};

struct node{
int x,y;
int h;
vector<pair<int,int>> path;
};

class priorityqueue{
public:
vector<node> pq;
void push(node n){
pq.push_back(n);
}
node pop(){
int minindex=0;
for(int i=1;i<pq.size();i++){
if(pq[i].h<pq[minindex].h){
minindex=i;
}
}
node best=pq[minindex];
pq.erase(pq.begin()+minindex);
return best;
}
bool empty(){
return pq.empty();
}
};

int heuristic(int x,int y,int gx,int gy){
return abs(gx-x)+abs(gy-y);
}

int main(){
vector<rule> rules={
{1,"Robot at Entry","Insert start node into OPEN list"},
{2,"OPEN list not empty","Select node with minimum heuristic"},
{3,"Selected node is Exit","Stop search and output path"},
{4,"Selected node is not Exit","Expand neighbors"},
{5,"Neighbor is Wall/Room","Ignore neighbor"},
{6,"Neighbor already visited","Ignore neighbor"},
{7,"Valid neighbor","Insert into OPEN list"},
{8,"OPEN list empty","Stop search (No path found)"}
};
cout<<"robot evacuation table"<<endl;
for(auto r : rules){
cout<<"rule "<<r.id<<":"<<endl;
cout<<"condition: "<<r.cond<<endl;
cout<<"action: "<<r.action<<endl;
}
vector<vector<int>> floor={
```

```
{0,0,0,1,0,0,0},
{1,1,0,1,0,1,0},
{0,0,0,0,0,1,0},
{0,1,1,1,0,0,0},
{0,0,0,0,1,1,0},
{0,1,0,0,0,0,0}
};
int rows=floor.size();
int cols=floor[0].size();
int sx=0,sy=0;
int gx=5,gy=6;
vector<vector<bool>> vis(rows,vector<bool>(cols,false));
priorityqueue pq;
node start;
start.x=sx;
start.y=sy;
start.h=heuristic(sx,sy,gx,gy);
start.path.push_back({sx,sy});
pq.push(start);
int exp=0;
int dx[4]={-1,1,0,0};
int dy[4]={0,0,-1,1};
cout<<"\n best_first_search evacuation"<<endl;
while(!pq.empty()){
node curr=pq.pop();
exp++;
int x=curr.x;
int y=curr.y;
if(vis[x][y]){
continue;
}
vis[x][y]=true;
if(x==gx && y==gy){
cout<<"evacuation_path"<<endl;
for(auto p : curr.path){
cout<<"("<<p.first<<","<<p.second<<")";
}
cout<<"\ncells explored:"<<exp<<endl;
return 0;
}
for(int i=0;i<4;i++){
int nx=x+dx[i];
int ny=y+dy[i];
if(nx>=0 && nx<rows && ny>=0 && ny<cols){
if(!vis[nx][ny] && floor[nx][ny]==0){
node next;
next.x=nx;
next.y=ny;
next.h=heuristic(nx,ny,gx,gy);
next.path=curr.path;
next.path.push_back({nx,ny});
pq.push(next);
```

```
}
}
}
}
cout<<"exit not reachable";
return 0;
}
```

OUTPUT:
robot evacuation table
rule 1:
condition: Robot at Entry
action: Insert start node into OPEN list
rule 2:
condition: OPEN list not empty
action: Select node with minimum heuristic
rule 3:
condition: Selected node is Exit
action: Stop search and output path
rule 4:
condition: Selected node is not Exit
action: Expand neighbors
rule 5:
condition: Neighbor is Wall/Room
action: Ignore neighbor
rule 6:
condition: Neighbor already visited
action: Ignore neighbor
rule 7:
condition: Valid neighbor
action: Insert into OPEN list
rule 8:
condition: OPEN list empty
action: Stop search (No path found)

best_first_search evacuation
evacuation_path
(0,0)(0,1)(0,2)(1,2)(2,2)(2,3)(2,4)(3,4)(3,5)(3,6)(4,6)(5,6)
cells explored:12